



PENGEMBANGAN DASHBOARD IoT Quality Control & Deteksi Cacat Produk Real-Time Berbasis

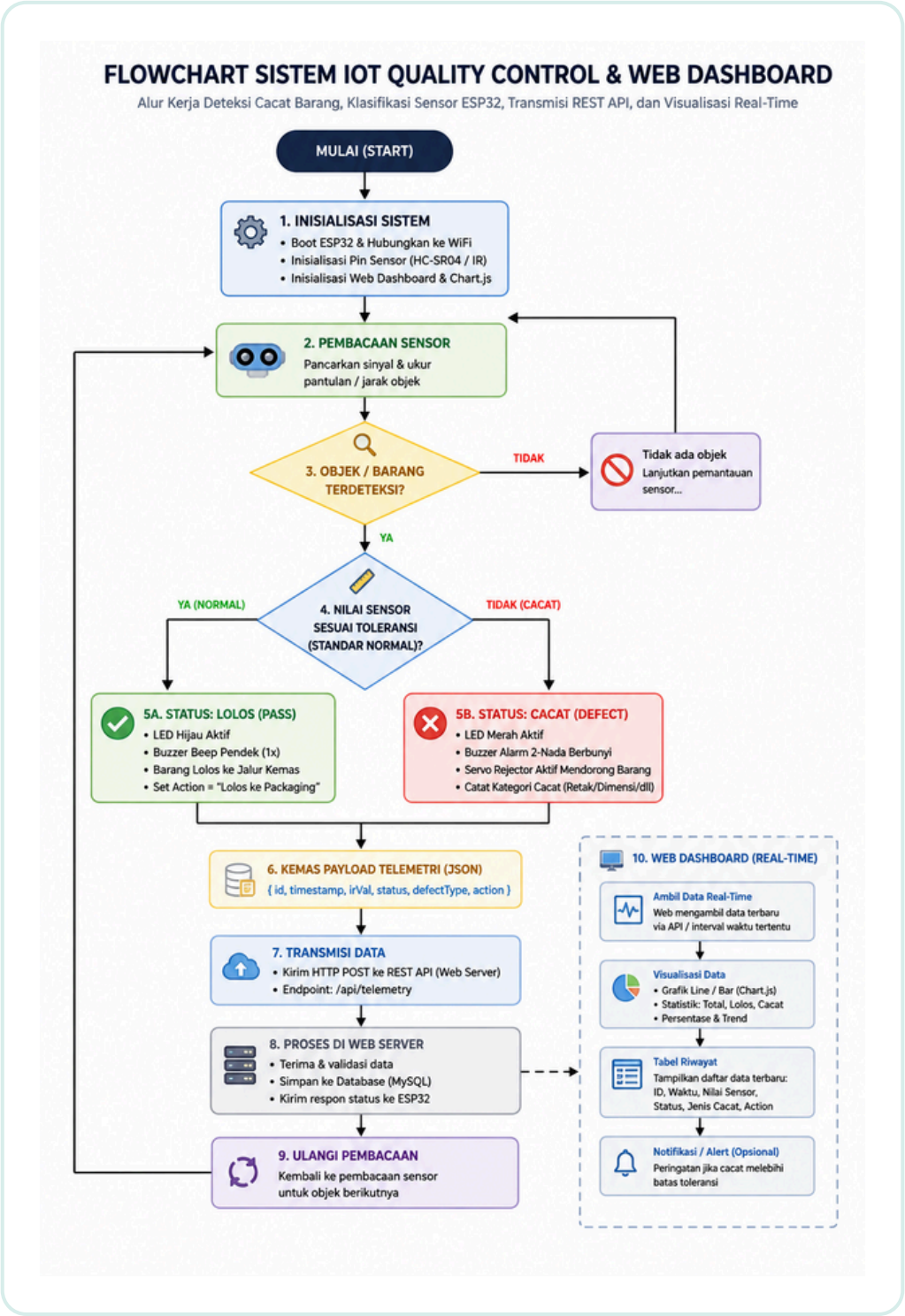
ESP32, SENSOR HC-SR04, WEB DASHBOARD,
DAN NODE-RED

KELOMPOK PENYUSUN

- Muhammad Haikal S - 2310631170101
- Mochammad Fadli - 2310631170161
- Dafa Alfauzi - 2310631170131
- Abdurrahman - 2310631170059
- Farhan Ahmad R - 2310631170017

Flowchart sistem IoT quality control

Gambaran menyeluruh alur kerja: deteksi objek, klasifikasi ESP32, transmisi REST API, hingga visualisasi real-time di web dashboard.



Mengapa sistem ini dibangun?

Dari inspeksi manual → monitoring otomatis

- Memvisualisasikan data inspeksi quality control secara real-time.
- Mengukur jarak/dimensi objek pada konveyor menggunakan ESP32 + HC-SR04.
- Mengklasifikasikan produk menjadi PASS atau DEFECT berdasarkan threshold.
- Menyediakan monitoring lintas perangkat: desktop, laptop, tablet, dan smartphone.
- Menghubungkan data ke Python REST API, Node-RED, dan Firebase Realtime Database.

TARGET SISTEM

01 **SENSE**
Sensor membaca objek

02 **DECIDE**
Threshold menentukan status

03 **VISUALIZE**
Dashboard tampilkan data

04 **ACT**
Alarm / reject action

Teknologi yang membentuk sistem

■ HARDWARE

- ESP32 Dev Module
- HC-SR04 Ultrasonic
- Onboard LED

■ FRONTEND

- HTML5
- Tailwind CSS
- Vanilla JS • Chart.js

■ BACKEND

- Python HTTPServer
- REST API
- serial_bridge.py

■ IOT TOOLS

- Node-RED
- Firebase Realtime DB
- ThingsBoard compatible

Modul Pengembangan Sistem IoT

Struktur 5 modul berkesinambungan dalam merancang & membangun sistem Quality Control dari nol hingga live monitoring

MODUL 01

Perakitan Hardware

Pondasi fisik akuisisi data sensor dimensi pada jalur konveyor pabrik

- Modul mikrokontroler
`ESP32 DevKit` 30-Pin
- Sensor ultrasonik `HC-SR04` 40 kHz
- Kabel DuPont & breadboard
- Pengkabelan daya `VIN 5V` & `GND`
- Jalur sinyal `Trig D5` & `Echo D19`
- Alignment tegak lurus (5–6 cm)

HASIL MODUL:

Rangkaian sirkuit sensor aktif dengan suplai daya USB stabil.

MODUL 02

Firmware & Edge AI

Pemrosesan data di mikrokontroler untuk klasifikasi kualitas seketika

- Sketch Arduino C++ teroptimasi
- Kalkulasi Time-of-Flight pulsa suara
- Algoritma thresholding:
PASS: 4.0 – 7.0 cm
DEFECT: <4.0 atau >7.0 cm
- Indikator LED GPIO 2 (kedip 3x)
- Serial JSON payload 115200 bps

HASIL MODUL:

ESP32 mandiri mengklasifikasi PASS/DEFECT & memancarkan JSON.

MODUL 03

Backend & Bridge

Penghubung komunikasi serial hardware ke ekosistem jaringan & web

- Web Serial API (1-klik koneksi direct browser tanpa driver)
- `server.py` REST API lokal port 3000
- `serial_bridge.py` auto-detect port
- Fitur Dual-Forwarding simultan
- `virtual_esp32.py` simulator

HASIL MODUL:

Pipeline data stream latensi rendah (<15 ms Web Serial, <45 ms Bridge).

MODUL 04

Cloud & Otomasi

Penyimpanan riwayat inspeksi & integrasi telemetri platform cloud

- Google Firebase Realtime DB
- Sinkronisasi path `/inspections`
- ThingsBoard Cloud IoT platform via Device Access Token
- Alur logika `node_red_flow.json`
- Routing multi-destinasi otomatis

HASIL MODUL:

Database NoSQL cloud tersinkronisasi real-time & multi-platform.

MODUL 05

Dashboard & UI

Antarmuka visual responsif bagi operator QC di desktop maupun mobile

- Dashboard SPA (HTML5 & Tailwind)
- Grafik Chart.js doughnut & line trend
- Dynamic Thresholding sliders (toleransi dapat diatur live)
- Metrik Throughput & Defect Rate
- Audio Alarm Buzzer & Export CSV

HASIL MODUL:

Dashboard interaktif real-time, responsive mobile, & kontrol batas toleransi.

Step-by-Step Pembuatan: Hardware & Firmware

Langkah praktis perakitan komponen fisik, skema perkabelan pinout, dan penulisan logika mikrokontroler

1

TAHAP 1: WIRING PINOUT

Perakitan Komponen Fisik

Menghubungkan sensor HC-SR04 ke board mikrokontroler ESP32 DevKit V1

- Pasang modul ESP32 dan sensor HC-SR04 pada breadboard uji.
- Hubungkan jalur catu daya:
`VCC HC-SR04` → `VIN (5V) ESP32`
`GND HC-SR04` → `GND ESP32`
- Hubungkan pin sinyal sonar:
`Trig Pin` → `GPIO 5 (D5)`
`Echo Pin` → `GPIO 19 (D19)`
- Posisikan sensor tegak lurus 5.0–6.0 cm di atas bidang konveyor.

KUNCI KEBERHASILAN:

Pastikan kontak jumper rapat dan bebas hambatan kerucut pantul ($\pm 15^\circ$).

2

TAHAP 2: LOGIKA FIRMWARE

Koding Pembacaan Jarak

Menerapkan algoritma Time-of-Flight gelombang suara ultrasonik 40 kHz

- Kirim pulsa LOW 2 μ s disusul HIGH 10 μ s pada pin `Trig` untuk memancarkan sonar.
- Gunakan `pulseIn(ECHO_PIN, HIGH)` untuk menangkap durasi waktu pantulan.
- Konversi durasi mikrodetik ke satuan jarak centimeter fisik:
$$\text{jarak} = (\text{durasi} * 0.0343) / 2$$
- Filter batas jarak jangkauan (>20 cm diabaikan / idle state).

KUNCI KEBERHASILAN:

Akurasi pengukuran presisi hingga ± 0.1 cm tanpa library eksternal.

3

TAHAP 3: EDGE CLASSIFICATION

Klasifikasi & Feedback Fisik

Evaluasi otomatis batas toleransi ketebalan/jarak barang di sisi mikrokontroler

- **Kondisi Lolos (PASS):**
Jarak 4.0 cm s.d. 7.0 cm (barang normal). LED onboard GPIO 2 menyala 1x sejenak.
- **Kondisi Cacat (DEFECT):**
Jarak <4.0 cm (terlalu tebal/menonjol) atau >7.0 cm (penyok/cekung). LED GPIO 2 berkedip cepat 3x.
- Bungkus hasil evaluasi menjadi objek JSON terstruktur.

KUNCI KEBERHASILAN:

Keputusan lolos/cacat diambil instan di level perangkat dalam waktu <2 ms.

4

TAHAP 4: FLASHING & SERIAL

Upload & Validasi Serial

Pengunggahan firmware via USB dan validasi aliran data telemetri

- Buka Arduino IDE, pilih board target *DOIT ESP32 DEVKIT V1*.
- Atur baud rate port serial pada `115200 bps`.
- Upload sketch `esp32_ultrasonic_defect_detector.ino`.
- Buka *Serial Monitor* dan uji sampel benda fisik: verifikasi baris JSON memuat atribut `id`, `distance`, dan `status`.

KUNCI KEBERHASILAN:

Data JSON terpancar kontinyu dan stabil siap dibaca lapisan software.

Step-by-Step Pembuatan: Cloud & Dashboard

Langkah praktis integrasi pipeline backend, konfigurasi cloud database, dan peluncuran web dashboard

5

TAHAP 5: MIDDLEWARE

Backend & Bridge Python

Membangun server lokal dan perute data serial COM otomatis

- Jalankan backend server HTTP lokal:
`python server.py` (Port 3000).
Menyediakan REST endpoint
`/api/inspection` & `/api/latest`.
- Jalankan bridge serial port:
`python serial_bridge.py`.
Mendeteksi port COM USB secara otomatis dan meneruskan payload via *dual-forwarding* simultan.
- (Uji Coba Virtual): Jalankan `virtual_esp32.py` bila menguji tanpa hardware fisik.

KUNCI KEBERHASILAN:

Server melayani akses web sekaligus menerima stream inspeksi tanpa konflik port.

6

TAHAP 6: CLOUD IOT

Integrasi Firebase & ThingsBoard

Menyimpan riwayat inspeksi & telemetry pada cloud platform industri

- **Google Firebase Realtime DB:**
Buat database cloud (region Asia), atur rules `.read: true, .write: true`, lalu salin URL database ke pengaturan web dashboard untuk logging real-time di path `/inspections`.
- **ThingsBoard Cloud:**
Buat entity device `esp32-qc`, salin *Device Access Token*, dan import file template `thingsboard_dashboard.json`.

KUNCI KEBERHASILAN:

Data otomatis tersimpan di cloud NoSQL dan visual ThingsBoard aktif.

7

TAHAP 7: AUTOMATION FLOW

Alur Pipeline Node-RED

Orkestrasi alur data visual multi-tujuan tanpa coding ulang firmware

- Jalankan runtime Node-RED di terminal:
`npx node-red` (Akses: `localhost:1880`).
- Klik menu → *Import* → pilih berkas `node_red_flow.json` dari folder proyek.
- Alur akan menghubungkan node *HTTP In (/api/inspection)* → *Function Evaluator* → *Firebase REST Out & Dashboard UI Gauge*.
- Klik tombol merah **Deploy**.

KUNCI KEBERHASILAN:

Alur visual mendistribusikan data ke panel telemetry dan database secara terpisah.

8

TAHAP 8: DASHBOARD & TUNING

Peluncuran Web Dashboard

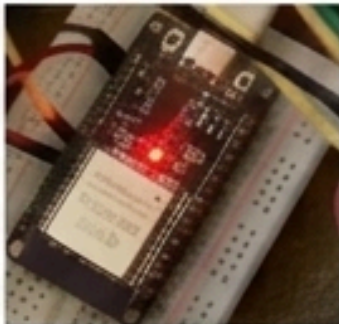
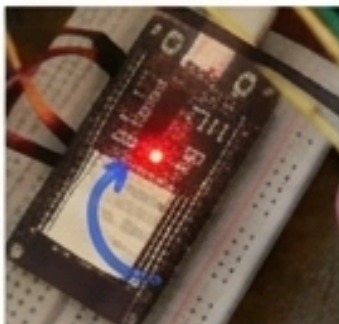
Monitoring operasional, grafik interaktif, dan kalibrasi dinamis

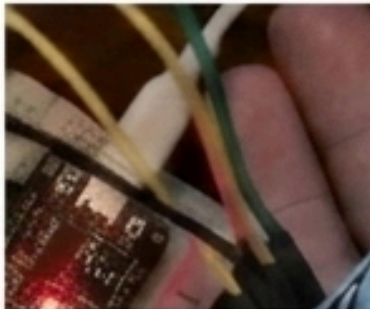
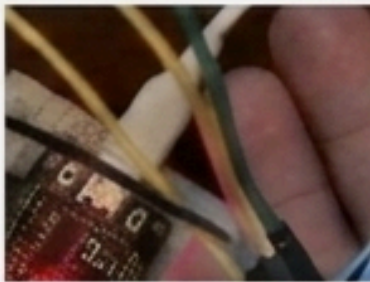
- Buka browser (Chrome/Edge) di `http://localhost:3000`.
- Gunakan tombol [ **Sambungkan ESP32 USB**] via Web Serial API (koneksi instan).
- Amati visualisasi *Chart.js* (grafik donat rasio dan tren sinyal sonar) serta kartu metrik *Throughput & Defect Rate*.
- Uji **Dynamic Threshold Slider** untuk mengubah batas toleransi min/max secara live tanpa reflash ESP32.

KUNCI KEBERHASILAN:

Dashboard responsif sempurna di desktop & smartphone dengan audio alarm aktif.

KOMPONEN HARDWARE

No	Nama Komponen	Penjelasan	Gambar Komponen
1	ESP32 DevKit V1 (Dev Module)	Mikrokontroler Utama Prosesor Dual-Core Tensilica Xtensa 32-bit LX6, hingga 240 MHz. Dilengkapi Wi-Fi 802.11 b/g/n dan Bluetooth v4.2 BR/EDR/BLE. Bertugas membaca sensor, menjalankan logika klasifikasi, dan mengirim data ke komputer.	 ESP32 DevKit V1 pada breadboard
2	Sensor Ultrasonik HC-SR04	Sensor Jarak / Dimensi Mengukur jarak/dimensi objek menggunakan gelombang ultrasonik 40 kHz (metode time-of-flight). Memiliki 4 pin: VCC, GND, Trig, dan Echo.	 Sensor HC-SR04
3	LED Indikator Onboard (GPIO 2)	Indikator Status LED bawaan ESP32 yang menyala sesaat saat barang berstatus PASS. Berkedip 3x saat barang terdeteksi DEFECT.	 LED Indikator menyala (titik merah)

4	Kabel Jumper DuPont (Female-Female)	Penghubung Sinyal & Daya Menghubungkan pin-pin sensor HC-SR04 ke pin GPIO ESP32. Digunakan untuk jalur VCC, GND, Trig, dan Echo.	 Kabel jumper warna-warni
5	Kabel Data USB (Micro-USB/Type-C)	Suplai Daya & Komunikasi Serial Menghubungkan ESP32 ke laptop/PC untuk daya sekaligus komunikasi serial. Digunakan untuk proses upload firmware dan streaming data.	 Pert USB-C pada ESP32

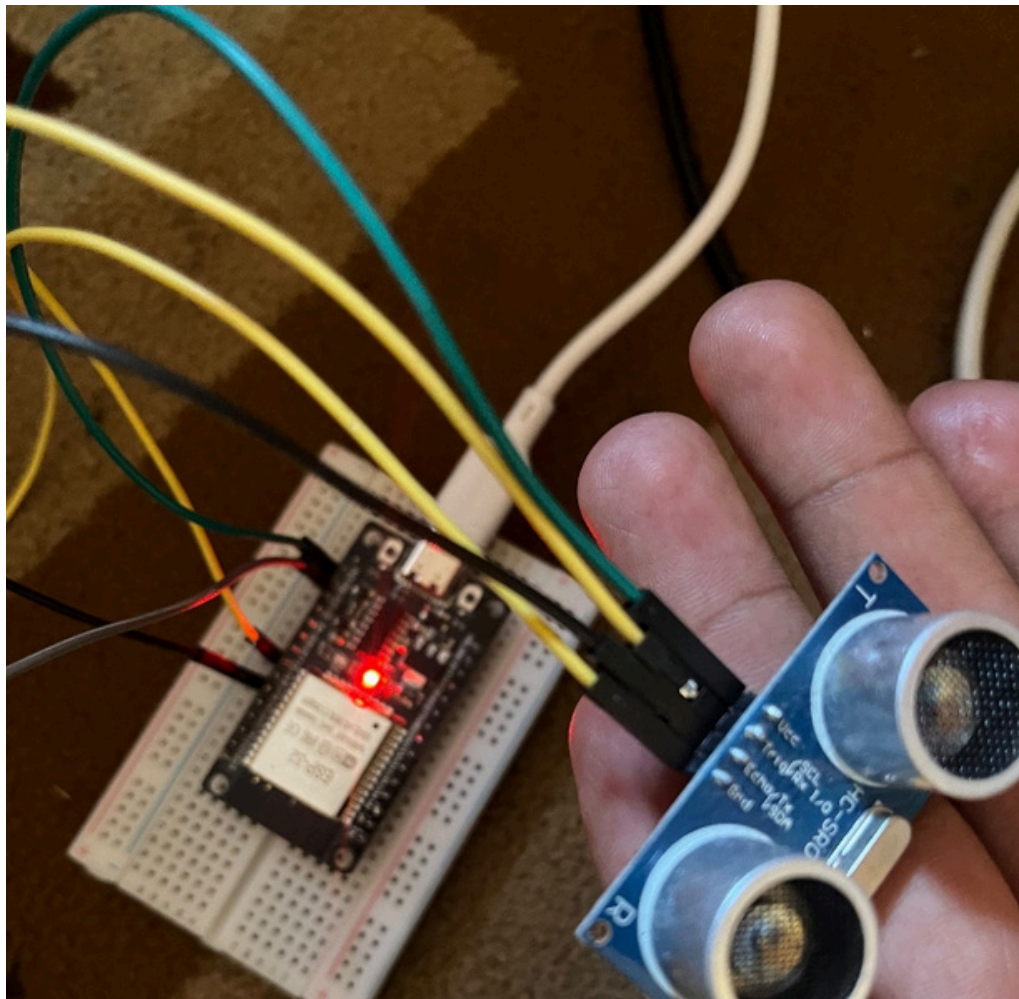
Wiring Diagram (Skema Pengkabelan)

No	Pin Sensor HC-SR04	Pin ESP32 DevKit	Fungsi
1	 VCC	 VIN (3.3V)	 Suplai daya utama sensor
2	 GND	 GND	 Ground / referensi tegangan bersama
3	 Trig (Trigger)	 GPIO 5 (D5)	 Memicu pemancaran pulsa ultrasonik
4	 Echo (Receiver)	 GPIO 19 (D19)	 Menerima/membaca durasi pantulan pulsa

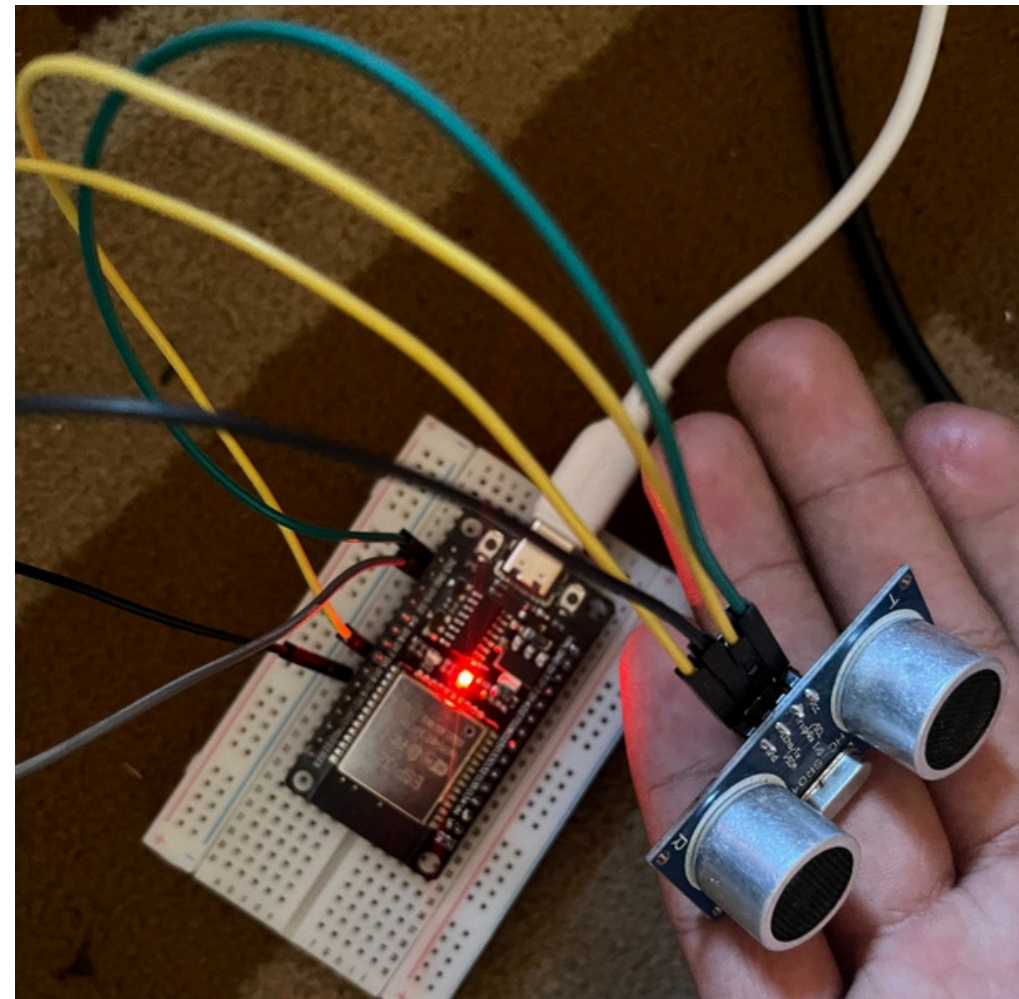
PEMASANGAN HARDWARE

Tahap	Koneksi Pin	Penjelasan
Penyiapan Alat & Komponen <i>Persiapan awal</i>	Tidak ada koneksi baru	Menyiapkan ESP32 DevKit V1, sensor HC-SR04, kabel jumper, dan kabel USB. Memeriksa seluruh pin header agar tidak longgar/short circuit.
Pemasangan Catu Daya <i>VCC & GND</i>	VCC → VIN/3.3V GND → GND	Kabel merah menghubungkan VCC sensor ke VIN/3.3V ESP32. Kabel hitam/cokelat menghubungkan GND sensor ke GND ESP32 sebagai common ground.

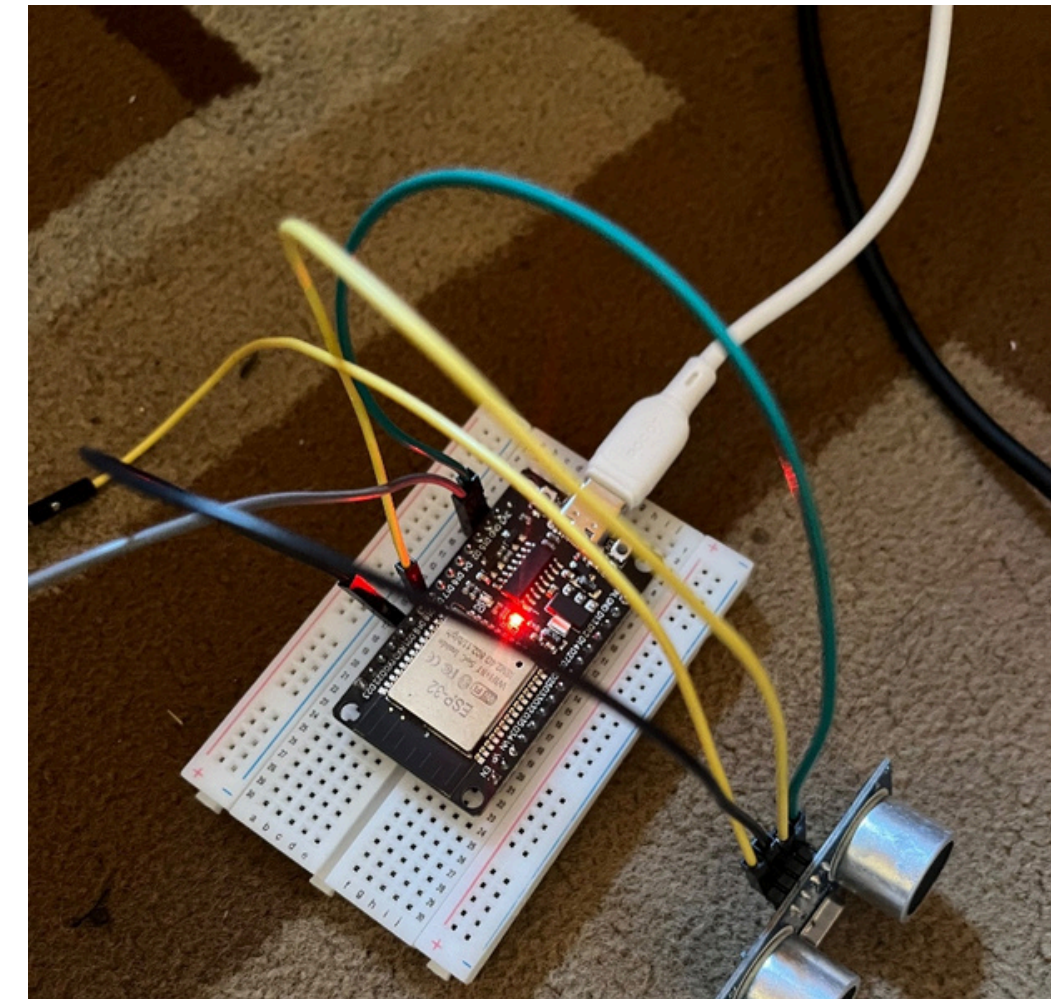
Bukti implementasi perangkat



ESP32 + HC-SR04



Sensor & wiring



Prototipe aktif

Pengujian dilakukan melalui USB; data sensor berhasil mengalir ke dashboard dan mendukung monitoring real-time.

IMPLEMENTASI PROGRAM

A. Firmware ESP32 (esp32_ultrasonic_defect_detector.ino)

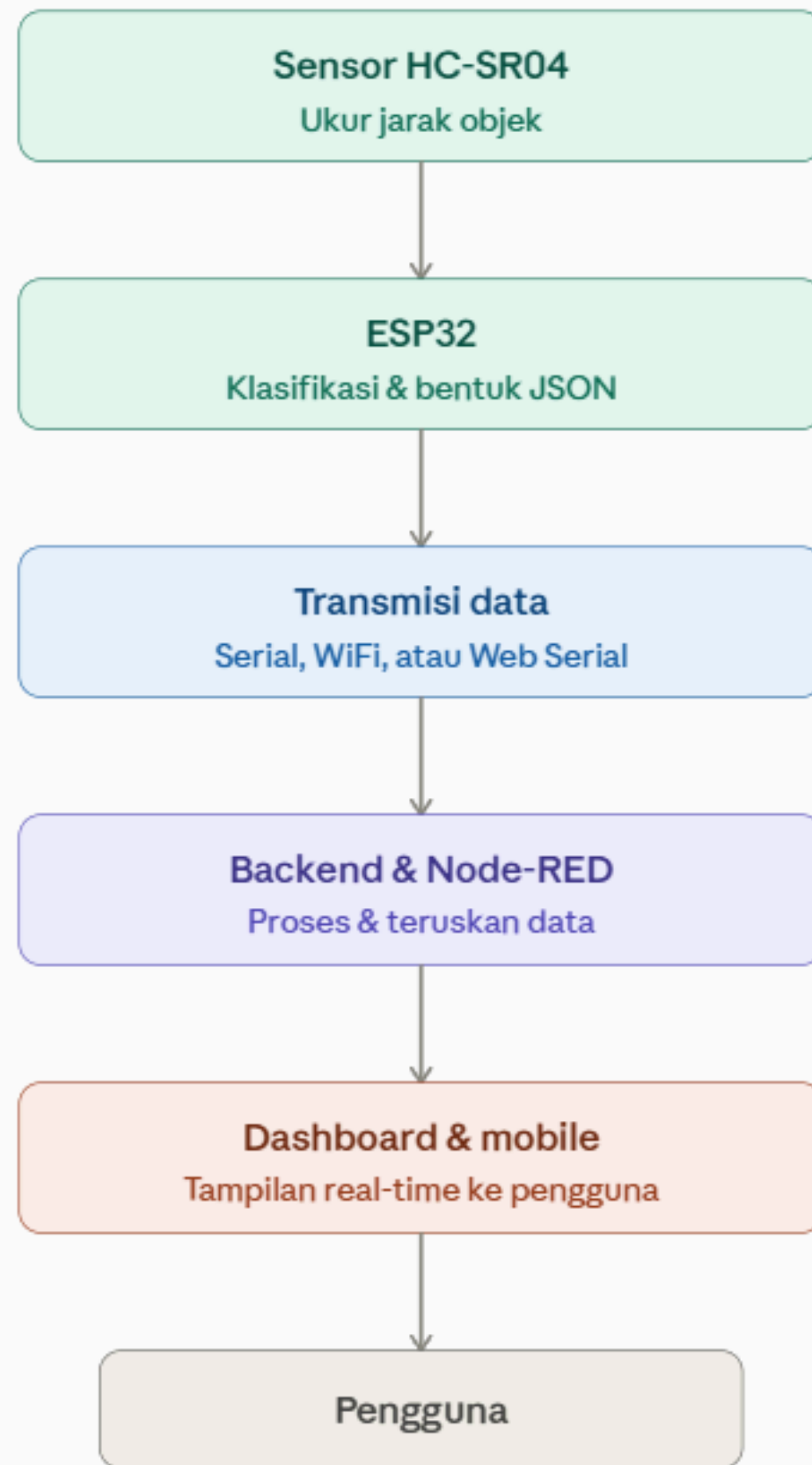
No	Bagian Program	Cuplikan Kode	Penjelasan
1	Library / Header <i>Pustaka yang digunakan</i>	<pre>// Tidak ada #include pustaka pihak ketiga // Fungsi sensor & komunikasi memakai // Arduino Core bawaan ESP32: // digitalWrite(), delayMicroseconds(), // pulseIn(), Serial.println()</pre>	Program <u>firmware</u> tidak memerlukan pustaka eksternal tambahan untuk sensor HC-SR04. Pengukuran jarak dilakukan secara <u>native</u> memakai fungsi bawaan <u>Arduino Core</u> ESP32. Komunikasi data ke komputer memakai pustaka <u>Serial</u> bawaan.
2	Pengaturan Pin <i>Definisi GPIO</i>	<pre>#define TRIG_PIN 5 #define ECHO_PIN 19 #define ONBOARD_LED 2</pre>	Menentukan <u>pin</u> GPIO yang dipakai: GPIO 5 untuk <u>Trig</u> (output pemicu), GPIO 19 untuk <u>Echo</u> (input pembaca pantulan), dan GPIO 2 untuk LED indikator <u>onboard</u> .
3	Inisialisasi Sensor <i>Ambang batas klasifikasi</i>	<pre>const float DETECTION_THRESHOLD_CM = 20.0; const float MIN_NORMAL_DIST_CM = 4.0; const float MAX_NORMAL_DIST_CM = 7.0;</pre>	Mendefinisikan konstanta ambang batas jarak sebagai acuan awal sensor: batas deteksi objek (20 cm) serta rentang dimensi normal produk (4.0–7.0 cm).
4	Proses Pembacaan Sensor <i>Fungsi measureDistance()</i>	<pre>float measureDistance() { digitalWrite(TRIG_PIN, LOW); delayMicroseconds(2); digitalWrite(TRIG_PIN, HIGH); delayMicroseconds(10); digitalWrite(TRIG_PIN, LOW); long duration = pulseIn(ECHO_PIN, HIGH, 30000); if (duration == 0) return 999.0; return (duration * 0.0343) / 2.0; }</pre>	Mengirim pulsa <u>trigger</u> 10 mikrodetik ke sensor, lalu membaca lebar pulsa <u>Echo</u> dengan <u>pulseIn()</u> . Durasi pulsa dikonversi menjadi jarak (cm) menggunakan rumus kecepatan suara (0.0343 cm/μs) dibagi 2.
5	Klasifikasi & Pengiriman Data <i>loop() → Serial.println()</i>	<pre>void loop() { float distance = measureDistance(); if (distance > 0.5 && distance <= DETECTION_THRESHOLD_CM) { ... String jsonPayload = "{\"id\":...}"; Serial.println(jsonPayload); } delay(300); }</pre>	Membentuk data hasil klasifikasi (PASS/DEFECT) menjadi objek JSON, lalu mengirimkannya melalui <u>Serial.println()</u> ke komputer untuk diteruskan ke <u>dashboard</u> .

IMPLEMENTASI PROGRAM

B. Backend Python — Serial Bridge (serial_bridge.py)

No	Bagian Program	Cuplikan Kode	Penjelasan
1	Library / Header <i>Pustaka Python</i>	<pre>import serial import serial.tools.list_ports import json import time import urllib.request</pre>	Pustaka pyserial untuk komunikasi port COM beserta list_ports untuk deteksi port otomatis, json untuk parsing payload data, time untuk jeda waktu penanganan koneksi, dan urllib.request untuk mengirim data HTTP POST.
2	Pembacaan Data Serial <i>serial_bridge.py</i>	<pre>try: ser = serial.Serial(target_port, baud_rate, timeout=1) time.sleep(2) print(f"✓") if line: if line.startswith('{') and line.endswith('}'): try: data = json.loads(line)</pre>	Menginisialisasi koneksi serial ke target port dengan jeda stabilisasi, lalu secara looping membaca data dari ESP32, memvalidasi format string JSON, memilahnya menjadi objek Python, dan menampilkan detail atribut sensor (ID, status, jarak, jenis cacat) di terminal.
3	Pengiriman / Penampilan Data <i>Dual-Forwarding</i>	<pre>reg_web = urllib.request.Request("http://localhost:3000/api/inspection", data=line.encode('utf-8'), headers={'Content-Type': 'application/json'}) urllib.request.urlopen(reg_web, timeout=2) reg_nr = urllib.request.Request("http://localhost:1880/api/inspection", ...) urllib.request.urlopen(reg_nr, timeout=2)</pre>	Meneruskan data hasil inspeksi secara simultan ke dua tujuan: Web Dashboard (server.py, endpoint /api/inspection) dan Node-RED Flow Engine, sehingga data dapat ditampikan secara real-time di kedua platform.

PROSES SAMPAI MENGHASILKAN DATA



1. Sensor HC-SR04 → mengukur jarak objek

Gelombang ultrasonik 40 kHz dipancarkan dan dipantulkan objek di jalur konveyor. Durasi pantulan dibaca lewat pin Echo, lalu dikonversi jadi jarak (cm) dengan rumus *time-of-flight*.

2. ESP32 → klasifikasi & bentuk JSON

ESP32 membandingkan jarak terukur dengan ambang batas (4.0–7.0 cm = normal). Hasilnya diklasifikasikan **PASS** atau **DEFECT**, lalu dibungkus jadi payload JSON (`id`, `status`, `distance`, `defectType`, `action`, `timestamp`).

3. Transmisi data → 3 metode paralel

Data JSON dikirim keluar dari ESP32 lewat salah satu dari 3 jalur:

- **Web Serial API** — koneksi 1-klik langsung dari browser (`< 15 ms`)
- **USB Serial → Python** `serial_bridge.py` — paling stabil, sekaligus terhubung ke Node-RED (`< 45 ms`)
- **WiFi HTTP POST** — ESP32 kirim langsung ke server nirkabel, cocok untuk instalasi pabrik (`< 95 ms`)

4. Backend & Node-RED → proses & teruskan data

`server.py` menerima data lewat endpoint REST API `/api/inspection`, menyimpan state terbaru (`/api/latest`). Secara paralel, Node-RED menerima payload yang sama untuk otomasi event, dan dapat meneruskannya ke **Firestore Realtime Database** (`/inspections`) untuk sinkronisasi cloud.

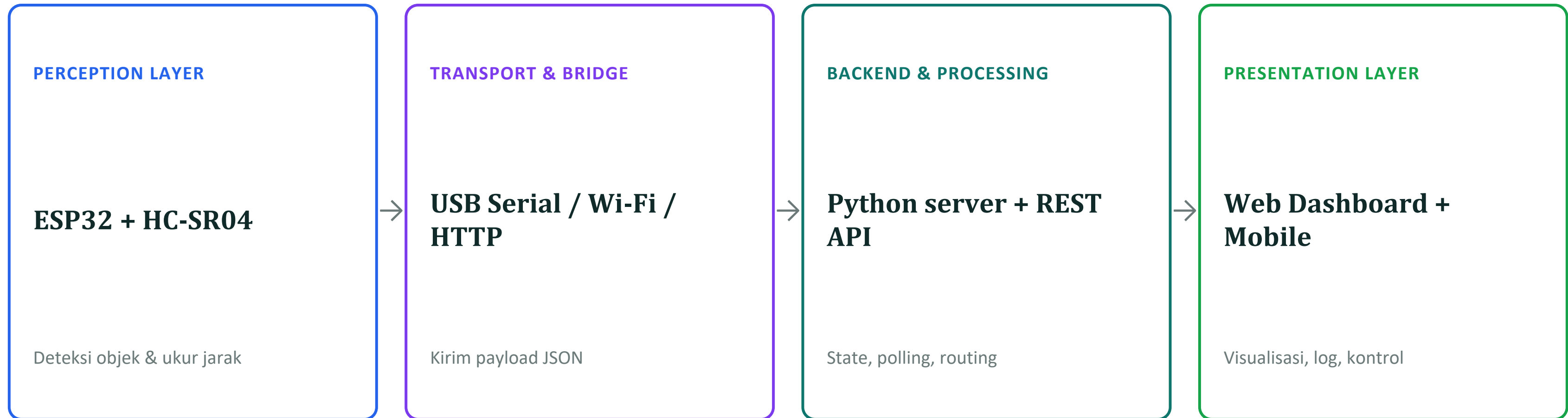
5. Dashboard & Mobile → tampilan real-time ke pengguna

Data yang sudah diproses ditarik oleh `app.js` dan divisualisasikan lewat Chart.js: Doughnut Chart (rasio PASS vs DEFECT), Line Chart (tren jarak 10 item terakhir), metric cards, dan tabel log inspeksi. Karena UI dibangun responsif (Tailwind CSS, Mobile-First), tampilan yang sama otomatis menyesuaikan di desktop, tablet, maupun smartphone.

6. Pengguna

Teman-teman satu kelompok kami atau pengguna lain dapat melihat data secara real time di layar baik di komputer maupun di hp.

Arsitektur end-to-end



Payload inspeksi dikirim sebagai JSON sehingga setiap layer dapat bertukar data dengan format yang konsisten.

Empat pilar penyajian data pada sistem IoT ini

Data mentah dari sensor hanya berguna jika sampai ke pengguna dengan cara yang tepat. Berikut peran masing-masing konsep dalam sistem ini.



Data Visualization

Mengubah angka jarak/status mentah dari sensor menjadi grafik (doughnut, trend line) dan kartu statistik, sehingga pola PASS/DEFECT langsung terlihat tanpa membaca log satu per satu.



Web Dashboard

Antarmuka terpusat berbasis HTML/Chart.js yang menerima data via REST API. Menjadi satu sumber kebenaran (single source of truth) untuk seluruh tim QC memantau status produksi.



Mobile Monitoring

Dashboard dirancang responsive (375px → 768px+) agar operator dapat memantau proses inspeksi dari tablet atau smartphone saat berada di lantai produksi, tidak hanya dari komputer.

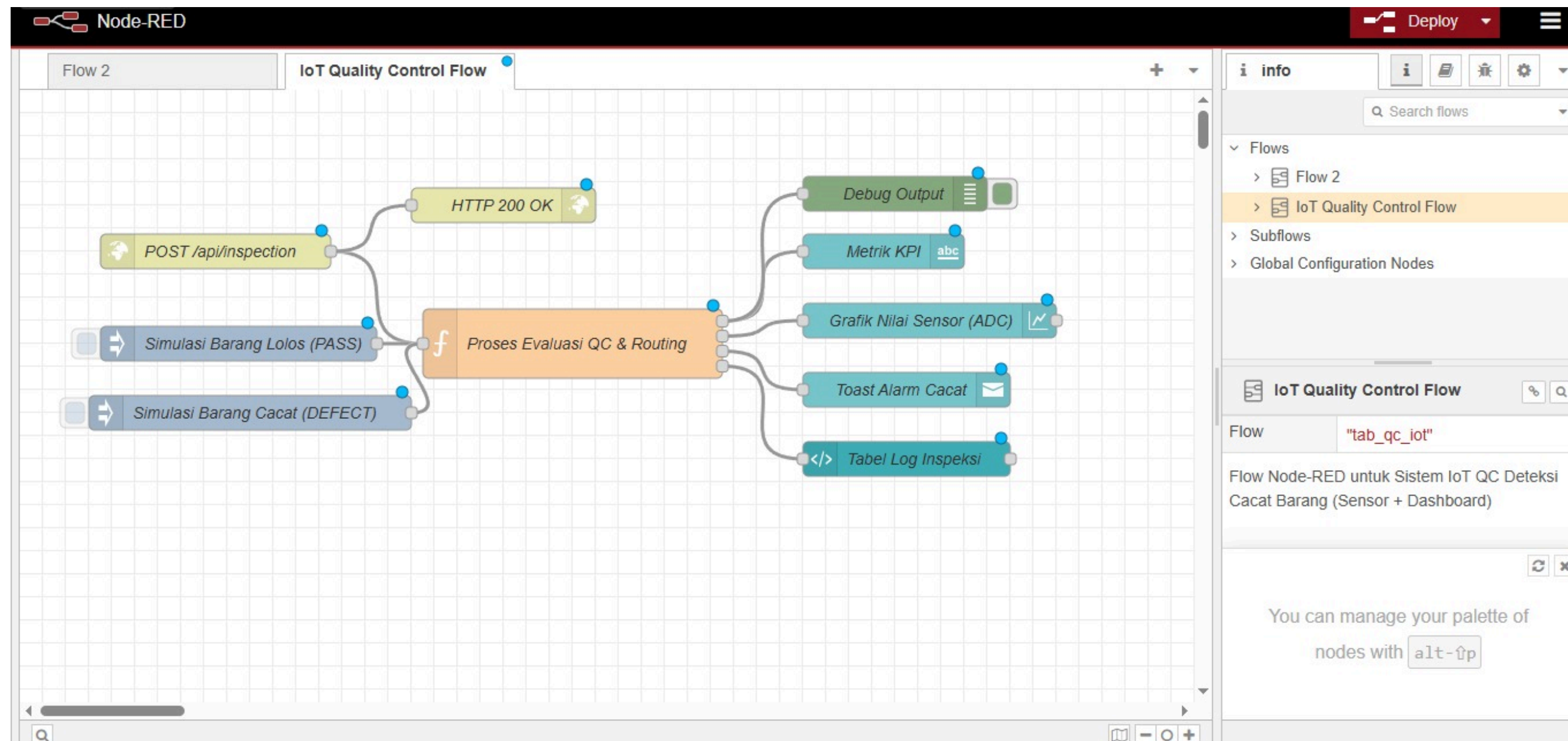


Real-time Data

ESP32 mengirim payload JSON secara terus-menerus melalui HTTP/Serial ke server, lalu dashboard melakukan polling/refresh berkala sehingga status terbaru tampil hampir seketika (<100 ms).

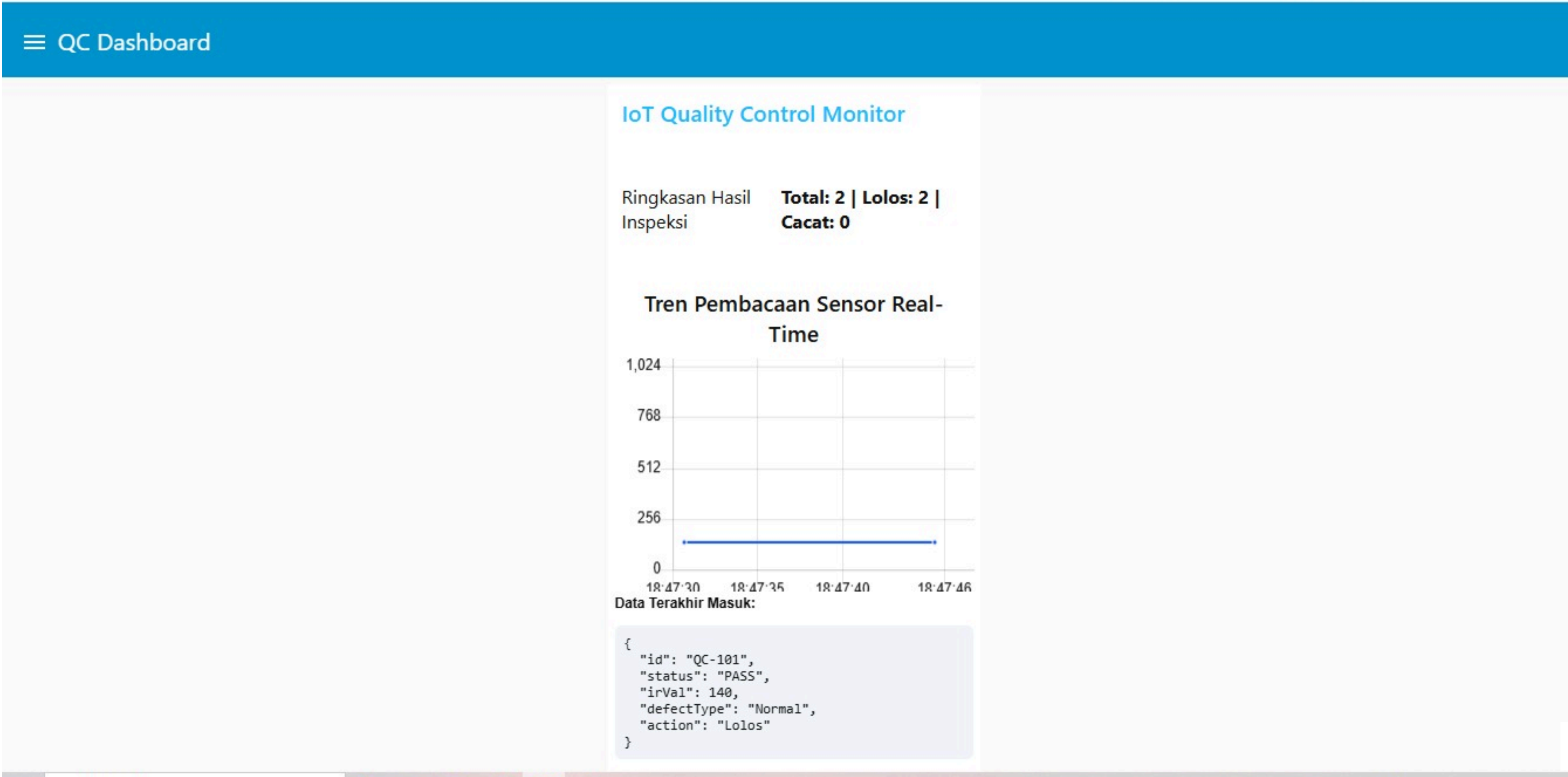
Tampilan alur Node-RED dashboard

Ilustrasi skematik flow editor Node-RED yang menerima payload JSON dari server Python, mengevaluasi status, lalu meneruskannya ke Firebase dan panel dashboard.



Node-RED merutekan satu payload ke banyak tujuan sekaligus — gauge, grafik tren, penyimpanan Firebase, dan notifikasi — tanpa mengubah kode di sisi ESP32.

Dashboard: dari data mentah menjadi insight



Integrasi Firebase

Database Cloud NoSQL untuk menyimpan riwayat inspeksi QC (/inspections) dan menyinkronkan data ke web secara real-time.

Alur Koneksi & Pengiriman Data

1. Node-RED Memproses Data

Node-RED mengevaluasi status barang (PASS/DEFECT) dari sensor, lalu meneruskan data JSON tersebut ke Firebase via REST API.

2. Firebase Menyimpan Log

Firebase menerima data dan menyimpannya secara rapi di dalam path */inspections*. (mencakup ID, status, jarak, dan *timestamp*).

3. Web Terupdate Otomatis

Web mendeteksi data baru di Firebase secara real-time, sehingga tampilan monitor langsung berubah tanpa perlu di-refresh.

<https://iot-defect-detection-1acb8-default-rtdb.asia-southeast1.firebaseio.com>

<https://iot-defect-detection-1acb8-default-rtdb.asia-southeast1.firebaseio.com/>

inspections

- ▶ -P0VRah1HBUEaYMLGYsh
- ▶ -P0VSQBKFw8u1V5EeD3q
- ▶ -P0VSXIXG-pG_7Ip5oAU
- ▶ -P0VS_xrpkZmVlflFjce
- ▶ -P0VT605-AIu3WCvy3iZ

Integrasi ThingsBoard Cloud

Platform IoT Cloud yang digunakan untuk device management, visualisasi dashboard, dan monitoring telemetry real-time

Alur Koneksi & Pengiriman Data

1. ESP32 Terhubung via Access Token

Perangkat atau simulator terhubung ke ThingsBoard Cloud menggunakan protokol HTTP/MQTT dengan mengautentikasi Access Token khusus milik perangkat yg dibuat.

2. Pengiriman Data Telemetry

Data pembacaan sensor (seperti parameter distance) dikirimkan secara berkala dalam format JSON langsung ke server ThingsBoard Cloud.

3. Visualisasi Dashboard Real-Time

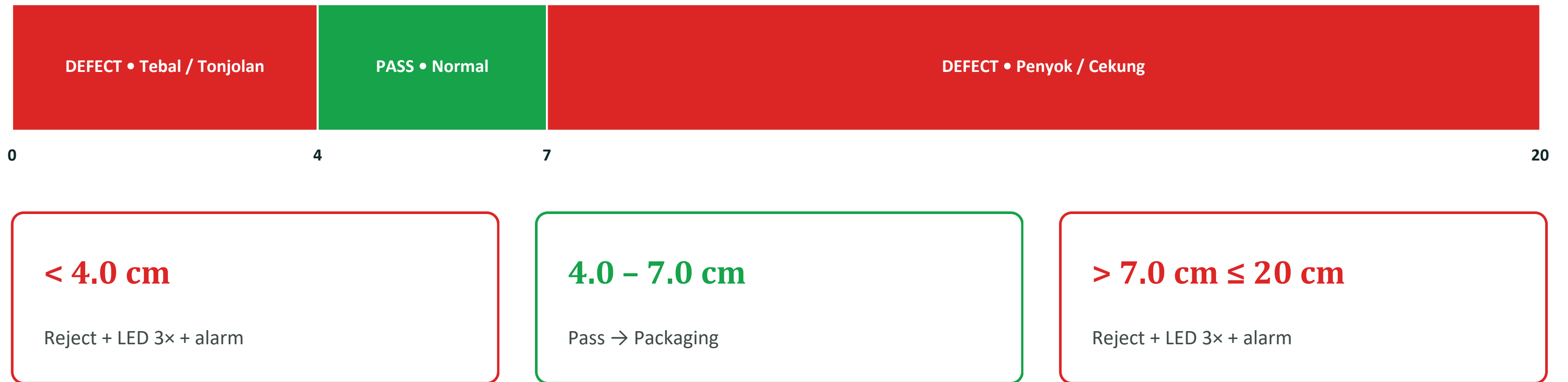
ThingsBoard mengolah data telemetry tersebut melalui Entity Alias yang sudah dikonfigurasi, sehingga grafik (Line Chart) dan indikator di dashboard langsung ter-update secara otomatis saat ada data baru.

Line chart



Logika deteksi: PASS vs DEFECT

Jarak sensor (cm)



Threshold ini dapat diubah dinamis lewat slider di dashboard, sehingga tim QC dapat menyesuaikan toleransi tanpa mem-flash ulang firmware ESP32.

Hasil pengujian: cepat, stabil, akurat

< 15 ms

WEB SERIAL

Sangat cepat

< 45 ms

PYTHON + BRIDGE

Sangat stabil

< 95 ms

WI-FI HTTP

Nirkabel penuh

100%

AKURASI KLASIFIKASI

Sampel pengujian

SAMPEL FISIK

ID	Jarak	Ekspektasi	Terdeteksi	Aksi
BRG-101	5.2 cm	PASS	PASS	Lolos → Packaging
BRG-102	5.5 cm	PASS	PASS	Lolos → Packaging
BRG-103	2.3 cm	DEFECT	DEFECT	Rejector + LED 3× + Alarm
BRG-104	6.1 cm	PASS	PASS	Lolos → Packaging
BRG-105	11.4 cm	DEFECT	DEFECT	Rejector + LED 3× + Alarm

Pengujian Visualisasi & Mobile

12 ms

Update Grafik Real-time

Visualisasi Chart.js merespons pembaruan data rata-rata 12 milidetik setelah payload diterima browser.

375–768 px

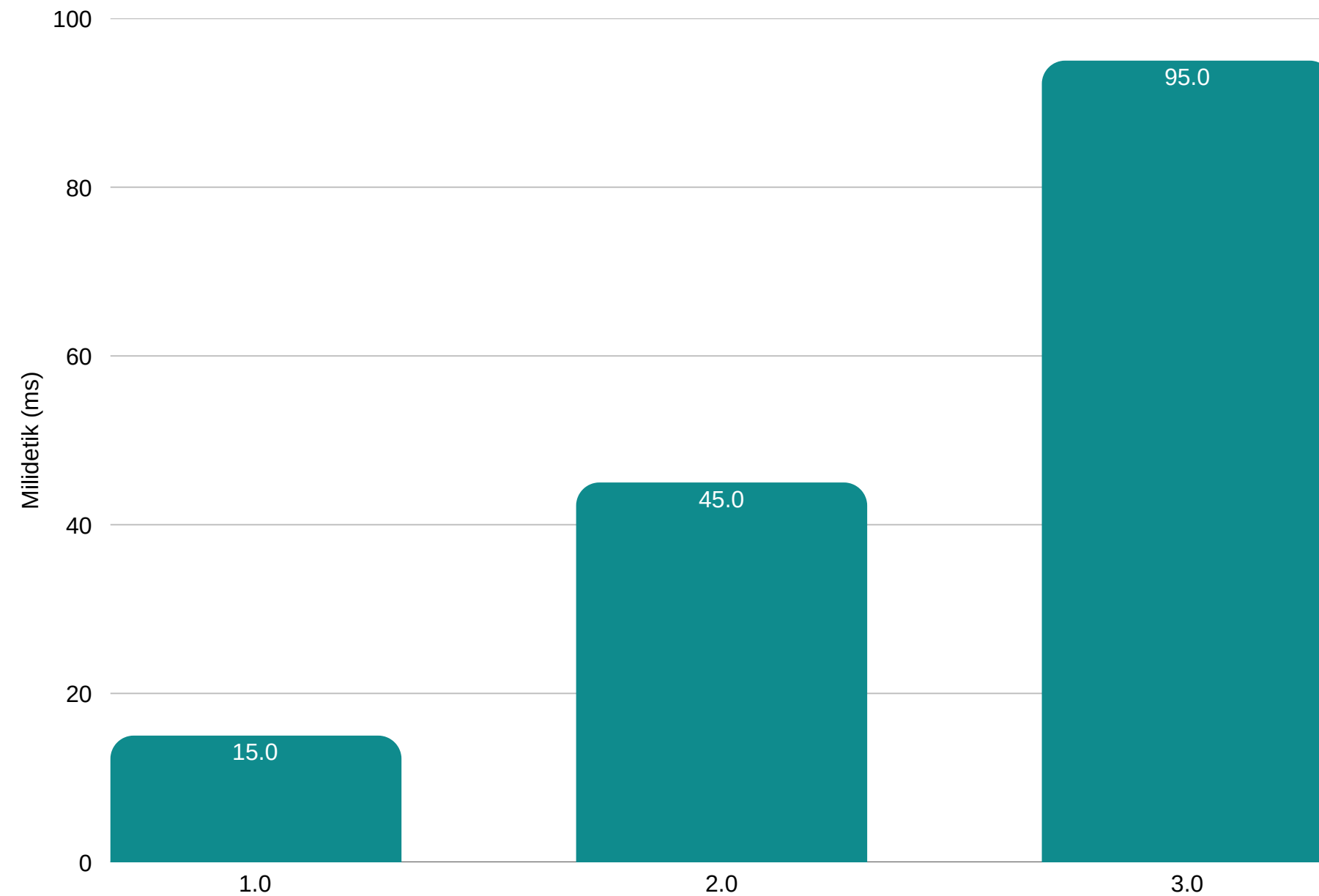
Responsivitas Mobile

Diuji pada layar smartphone (375px) & tablet (768px): kartu metrik tersusun rapi, tombol touch-friendly, grafik proporsional tanpa overflow.

Pengujian Konektivitas & Latensi Koneksi ESP32

Diuji menggunakan 3 metode transmisi data berbeda untuk membuktikan fleksibilitas dan keandalan sistem.

Rata-rata Latensi per Metode (ms)



< 15 ms

Web Serial API (1-Click)

Sangat cepat & tanpa perlu software tambahan

< 45 ms

Python Server + Bridge

Sangat stabil & terhubung ke Node-RED sekaligus

< 95 ms

WiFi HTTP Client

Nirkabel penuh, sangat cocok untuk instalasi pabrik

Kesimpulan & arah pengembangan

KESIMPULAN

- Dashboard IoT berhasil dikembangkan dan berfungsi stabil.
- Visualisasi real-time, web dashboard, dan mobile monitoring terpenuhi.
- Integrasi Web Serial, REST API Python, Node-RED, dan Firebase menunjukkan interoperabilitas luas.
- Akurasi klasifikasi pada sampel pengujian mencapai 100%.

Terima kasih • Tanya jawab

Referensi utama

1

Espressif Systems. (2024). ESP32 Series Datasheet & Technical Reference Manual.

2

Monk, S. (2020). Programming Arduino: Getting Started with Sketches. McGraw-Hill Education.

3

Node-RED Documentation. (2024). Flow-based Programming for the Internet of Things. OpenJS Foundation.

4

Google Firebase. (2024). Firebase Realtime Database Documentation. Google Developers.

5

Chart.js Community. (2024). Chart.js Open Source HTML5 Charts Documentation.

PEMBAGIAN TUGAS

- 1 DAFA ALFAUZI → Membuat kode json, arduino, html
- 2 ABDURRAHMAN → Membuat Laporan, revisi, dan handle bagian hardware dari dosen
- 3 M. HAIKAL SYAMSUDIN ---> wiring komponen bersama farhan, monitoring barang dan tugas tim
- 4 FARHAN AHMAD R → wiring komponen bersama haikal, revisi ppt, bikin kode server dan serial_bridge
- 5 MOCHAMMAD FADLI ----> Membuat Laporan, revisi, dan handle bagian hardware dari dosen